INSTITUTO FEDERAL
Espírito Santo
Campus Serra

REDES DE COMPUTADORES — PPCOMP

# Descoberta Dinâmica de Topologias em Redes Definidas por Software

## Mapeamento Ativo com LLDP e Gratuitous ARP utilizando Ryu e Mininet

---

**Alan Saar & Leonardo Herkenhoff**

Instituto Federal do Espírito Santo — Campus Serra | 2026

# Sumário

**1 Introdução e Contextualização do Paradigma SDN ..... 3**
**2 Descoberta de Switches e Enlaces Inter-Switch com LLDP ..... 4**
2.1 O Protocolo LLDP e o Mecanismo OFDP ..... 4
2.2 O Papel da Flag `--observe-links` e da API de Topologia do Ryu ..... 4
**3 Descoberta de Hosts e Contenção de Broadcast ..... 5**
3.1 O Mecanismo do Gratuitous ARP (GARP) ..... 5
3.2 Interceptação de Pacotes via `Packet-In` e Eliminação de Loops ..... 6
**4 Implementação dos Módulos de Software ..... 7**
4.1 Script Mininet: `teste_topologia.py` ..... 7
4.2 Controlador Ryu: `topo_discover.py` ..... 7
4.3 Gerador de Grafos: `export_topology_graph.py` ..... 8
**5 Resultados Experimentais e Validação de Consistência ..... 8**
5.1 Cenário 1: Topologia Triangle ..... 8
5.2 Cenário 2: Topologia Someloops ..... 11
**6 Reflexões Pedagógicas e Lições Aprendidas ..... 15**
**7 Conclusão ..... 15**

# 1 Introdução e Contextualização do Paradigma SDN

O advento das **Redes Definidas por Software** (*Software-Defined Networking* – SDN) representa uma quebra de paradigma fundamental na arquitetura das redes de computadores. Nas arquiteturas legadas, os planos de dados (*data plane*) e de controle (*control plane*) residem no interior do mesmo equipamento físico proprietário. Switches tradicionais aprendem endereços MAC de forma autônoma e executam protocolos distribuídos como o *Spanning Tree Protocol* (STP) para prevenir loops, desativando portas físicas redundantes de modo conservador.

Em contrapartida, a arquitetura SDN preconiza quatro preceitos basilares:

1. **Desacoplamento Físico e Lógico:** A inteligência da rede é extraída dos dispositivos de comutação e centralizada logicamente em uma entidade coordenadora denominada **Controlador SDN**.
2. **Switches Baseados em Fluxos:** Equipamentos de encaminhamento (comutadores OpenFlow) operam de forma reativa ou proativa com base em tabelas de fluxos (*Flow Tables*), realizando apenas o chaveamento de alta velocidade orientado pelo controlador.
3. **Visão Global e Centralizada:** O controlador atua como um Sistema Operacional de Rede (*Network Operating System* – NOS), mantendo uma representação abstrata completa de todos os nós e enlaces.
4. **Programabilidade por Software:** A rede é gerenciada programmaticamente por meio de interfaces abertas (*Southbound APIs*, como o OpenFlow 1.3).

> **O Dilema Fundamental da Descoberta em SDN**
>
> Em redes comutadas convencionais, switches operam de forma autônoma: eles escutam o tráfego de dados e alimentam tabelas de aprendizado de MAC (*MAC learning*). Em uma rede SDN pura, os comutadores são componentes sem inteligência própria de roteamento; ao ligar a alimentação, o switch OpenFlow desconhece completamente a quais vizinhos está conectado.
>
> Portanto, é responsabilidade indelegável do **Controlador SDN** sondar o plano de dados, injetar pacotes de descoberta e reconstruir o grafo da topologia física.

O presente trabalho técnico documenta a implementação de um sistema robusto de leitura, instanciação e descoberta dinâmica de topologias em SDN utilizando o framework **Ryu** (operando sobre OpenFlow 1.3) e o ambiente de emulação **Mininet**. O foco deste estudo concentra-se no mapeamento de enlaces inter-switch via pacotes **LLDP** e na localização passiva de estações hospedeiras (*hosts*) mediante **Gratuitous ARP**, com ênfase especial na contenção de tempestades de broadcast em topologias com múltiplos ciclos (**Triangle** e **Someloops**).

—

# 2 Descoberta de Switches e Enlaces Inter-Switch com LLDP

## 2.1 O Protocolo LLDP e o Mecanismo OFDP

Para mapear enlaces físicos entre switches sem intervenção manual, os controladores SDN empregam uma adaptação do protocolo **LLDP** (*Link Layer Discovery Protocol*, norma IEEE 802.1AB), frequentemente denominada na literatura técnica como **OFDP** (*OpenFlow Discovery Protocol*).

O protocolo LLDP opera estritamente na camada de enlace (EtherType `0x88CC`) e utiliza estruturas do tipo TLV (*Type-Length-Value*) para transportar dados de identificação, tais como o Chassis ID e o Port ID.

O ciclo operacional de descoberta via LLDP ocorre em quatro etapas sequenciais:

1. **Geração do Pacote pelo Controlador:** O Ryu gera um quadro Ethernet com EtherType `0x88CC`. No payload TLV, o Ryu insere o identificador único do switch de origem (`src_dpid`) e a porta de transmissão (`src_port_no`).
2. **Injeção via `Packet-Out`:** O controlador encapsula esse pacote em uma mensagem OpenFlow `OFPT_PACKET_OUT` e a envia ao switch de origem, instruindo-o a ejetar o quadro pela porta física sob sondagem.
3. **Transmissão no Cabo Físico:** O switch de origem transmite o quadro através do enlace em direção ao switch adjacente.
4. **Interceptação via `Packet-In`:** O switch vizinho recebe o quadro em uma de suas portas. Por possuir uma regra de fluxo que associa o EtherType `0x88CC` à ação `OFPP_CONTROLLER`, o switch não encaminha o pacote pela rede; em vez disso, encapsula o quadro em uma mensagem `OFPT_PACKET_IN` e o remete ao Ryu.

Ao inspecionar a mensagem `Packet-In`, o controlador correlaciona os metadados:

$$\text{Enlace Descoberto: Switch Origem } (s_A, p_A) \longrightarrow \text{Switch Destino } (s_B, p_B)$$

## 2.2 O Papel da Flag `--observe-links` e da API de Topologia do Ryu

No framework Ryu, o mecanismo de descoberta LLDP não fica ativo por padrão para economizar processamento. Ele é acionado através do módulo interno `ryu.topology.switches`, habilitado quando o executável do Ryu é inicializado com a flag:

```
1 ryu-manager --observe-links topo_discover.py
```

Uma vez habilitada, essa flag instancia um serviço em segundo plano que:

- Injeta pacotes LLDP periodicamente em todas as portas ativas de todos os switches conectados.
- Monitora os timers de expiração para detectar quedas de cabos físicos.
- Dispara os eventos da API interna do Ryu: `EventSwitchEnter`, `EventSwitchLeave`, `EventLinkAdd` e `EventLinkDelete`.

Na aplicação de monitoramento, podemos capturar esses eventos e invocar as funções `get_switch(self, None)` e `get_link(self, None)` da biblioteca `ryu.topology.api`, obtendo listas sincronizadas de todos os datalinks da rede.

> **Analogia Didática: O Galpão e os Funcionários (Luiz e Flávio)**
> Para ilustrar esse processo, imagine uma grande central de distribuição:
>
> - **O Gerente Geral (Controlador Ryu):** Senta-se em uma torre de controle enviadraçada com visão panorâmica, comunicando-se por rádio exclusivo com suas salas de triagem.
> - **Os Funcionários Luiz ($s_1$) e Flávio ($s_2$):** São operadores zelosos trancados em salas isoladas. Cada sala possui várias portas numeradas com tubos pneumáticos interligando as salas. Luiz e Flávio não sabem onde os tubos vão dar.
> - **A Flag `--observe-links`:** É a ordem matinal do Gerente: *"Equipe de mensageiros, ativem-se e comecem a soltar os bilhetes pelos tubos!"*.
> - **O Pacote LLDP:** O Gerente escreve em um papelzinho colorido: *"Sou da sala do Luiz, saí pela Porta 1"*. Pelo rádio (`Packet-Out`), o Gerente diz: *"Luiz, empurre este bilhete pelo tubo da Porta 1"*.
> - **A Comprovação do Enlace:** O bilhete viaja fisicamente pelo tubo e cai na sala de Flávio pela Porta 2. Flávio pega o rádio (`Packet-In`) e avisa o Chefe: *"Chefe, acabou de cair um bilhete na minha Porta 2 dizendo que veio da Porta 1 do Luiz!"*. O Gerente abre sua prancheta e anota o tubo entre Luiz e Flávio.

—

# 3 Descoberta de Hosts e Contenção de Broadcast

## 3.1 O Mecanismo do Gratuitous ARP (GARP)

Enquanto a infraestrutura de cabeamento entre switches é descoberta pelo LLDP, as estações de trabalho e servidores finais (*hosts*) não processam pacotes LLDP. Tradicionalmente, redes Ethernet descobrem máquinas no momento em que elas realizam requisições de broadcast (ARP Request) para localizar outros nós.

Entretanto, em redes SDN recém-iniciadas nas quais os hosts ainda não iniciaram comunicação mútua, faz-se necessária uma técnica ativa de anúncio de presença. A abordagem ideal é a emissão de pacotes **Gratuitous ARP** (GARP), gerados através da ferramenta de rede `arping`:

```
arping -U -c 1 <IP_DO_HOST>
```

Tecnicamente, o Gratuitous ARP é um quadro Ethernet do tipo `0x0806` cujo cabeçalho de Camada 2 possui endereço de destino em broadcast puro (`ff:ff:ff:ff:ff:ff`), mas cujo conteúdo interno na Camada 3 é autorreferencial:

$$\text{Sender IP} = \text{Target IP} = \text{IP do próprio Host}$$

## 3.2 Interceptação de Pacotes via `Packet-In` e Eliminação de Loops

Em switches não gerenciados convencionais, um quadro recebido com destino `ff:ff:ff:ff:ff:ff` é compulsoriamente retransmitido por todas as demais portas ativas (*flooding*). Caso a topologia física contenha ciclos fechados (como as topologias *Triangle* e *Someloops*), esse pacote entraria em replicação infinita, deflagrando uma **tempestade de broadcast** (*broadcast storm*) capaz de saturar os buffers e congelar a emulação.

No ambiente SDN implementado, essa vulnerabilidade é neutralizada na raiz através da **regra padrão de Table-Miss** instalada na tabela de fluxos (`Table 0`) dos comutadores:

```python
match = parser.OFPMatch()
actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
ofproto.OFPCML_NO_BUFFER)]
```

Quando o host injeta o Gratuitous ARP no cabo:

1. O switch OpenFlow recebe o frame na porta de acesso.
2. Como não há regra de fluxo instalada para aquele tráfego, o pacote atinge a regra **Table-Miss**.
3. O switch **abafa o broadcast**: ele não repassa o quadro para as demais portas físicas. Em vez disso, envia uma mensagem `OFPT_PACKET_IN` diretamente ao Ryu.
4. O controlador lê a carga útil da mensagem, extrai o endereço IP, o endereço MAC, o `datapath.id` do switch de entrada e a porta física `in_port`.
5. O host é registrado no dicionário `self.hosts` e o pacote é descartado sem gerar ruído na rede.

> **Analogia Didática: O Cartão de Visitas na Recepção**
> Continuando na analogia do galpão:

> - Quando um novo cliente chega à empresa (Host $h_1$), em vez de pegar um megafone e gritar pelos corredores em broadcast perturbando todos os setores, ele simplesmente entrega um **cartão de visitas não solicitado** (Gratuitous ARP) por debaixo da porta da sala de Luiz.
> - Luiz lê o cartão com o nome e o ramal do cliente, não repassa para os tubos vizinhos, coloca em um envelope confidencial e despacha diretamente ao Gerente Geral (`Packet-In`).
> - O Gerente Geral adiciona o cliente em seu livro de registros sem que ninguém no galpão tenha sofrido com poluição sonora ou tempestades de papéis.

—

# 4 Implementação dos Módulos de Software

A solução foi desenvolvida de maneira modular, com separação estrita de responsabilidades entre instanciação física no emulador, inteligência de controle no Ryu e renderização gráfica via NetworkX.

## 4.1 Script Mininet: `teste_topologia.py`

O script `teste_topologia.py` é responsável pela infraestrutura do plano de dados. Ele foi projetado para admitir grafos arbitrários expressos em formato de lista de arestas textual.

Destaques da implementação:

- **Parser Resiliente:** Lê arquivos de arestas, desprezando linhas em branco e comentários iniciados por # (permitindo manter diagramas conceituais em ASCII no cabeçalho do arquivo de entrada).
- **Endereçamento Determinístico:** Para cada switch $s_X$, instancia-se automaticamente um host $h_X$ configurado com IP `10.0.0.X/24` e MAC `00:00:00:00:00:0X`.
- **Sinalização GARP:** Imediatamente após `net.start()`, o script itera sobre todos os hosts e dispara o comando de transmissão não solicitada `arping -U -c 1`.
- **Automação sem CLI:** Suporte à flag `--no-cli`, permitindo execução em lote por scripts de teste contínuo.

## 4.2 Controlador Ryu: `topo_discover.py`

O controlador centraliza a lógica de mapeamento orientada a eventos sob OpenFlow 1.3:

- `@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)`: Conclui o aperto de mão e instala a regra **Table-Miss** prioritária nos switches recém-conectados.

- `@set_ev_cls(event.EventSwitchEnter)`: Recalcula os switches ativos invocando `get_switch()`.
- `@set_ev_cls(event.EventLinkAdd)`: Atualiza a matriz de adjacência de enlaces inter-switch através de `get_link()`.
- `@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)`: Filtra frames `arp.arp`, extrai as tuplas `(IP, MAC, DPID, Port)` e sincroniza a tabela de hosts.
- `print_full_topology()`: Consolida enlaces simétricos e gera relatórios limpos no log do console, evitando impressões duplicadas a cada pacote transitado.

### 4.3 Gerador de Grafos: `export_topology_graph.py`

Utilizando a biblioteca `networkx` e o backend gráfico do `matplotlib`, este script processa as arestas e exporta imagens em resolução de 300 DPI:

- Diferenciação cromática e geométrica: Switches como quadrados azuis com indicativo de DPID; Hosts como círculos verdes com endereço IP legível.
- Estilização de enlaces: Enlaces inter-switch LLDP com traço sólido espesso; enlaces de acesso a hosts com traço pontilhado.

—

## 5 Resultados Experimentais e Validação de Consistência

A validação prática foi conduzida em duas topologias complexas com múltiplos loops, avaliando a consistência entre o grafo físico instanciado e as estruturas descobertas dinamicamente pelo controlador.

### 5.1 Cenário 1: Topologia Triangle

A topologia **Triangle** consiste em um ciclo simples com 3 switches e 3 hosts conectados.

```
        h2 (10.0.0.2)
             |
             s2
            /  \
           /    \
          s1----s3
          |      |
          h1     h3
     (10.0.0.1)(10.0.0.3)
```

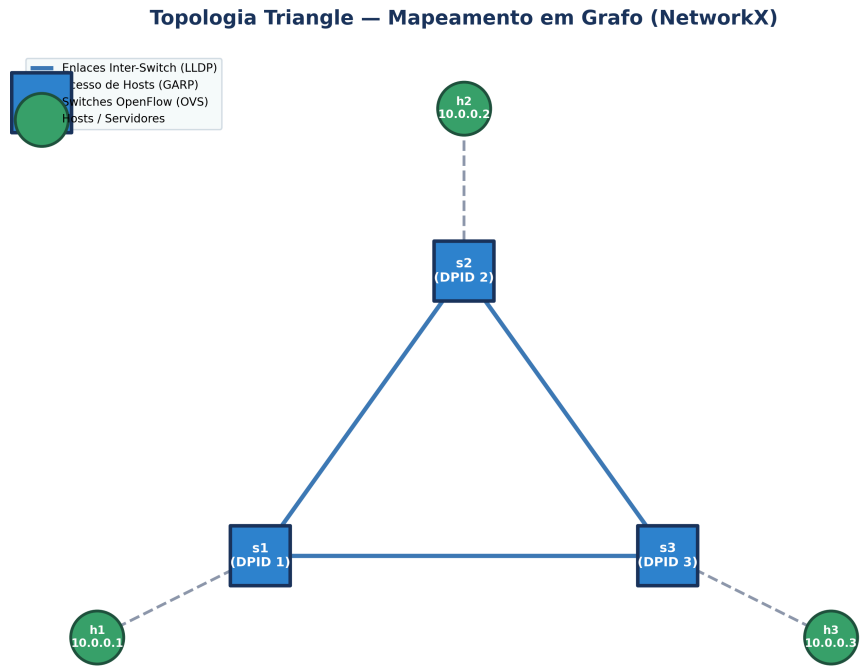



Figura 1: Topologia Triangle modelada em grafo NetworkX a partir das descobertas do Ryu.

### 5.1.1 Consistência Estrutural Mapeada

| Elemento | Especificação Teórica | Mapeamento Descoberto pelo Ryu | Status |
|---|---|---|---|
| Switches Ativos | $s_1, s_2, s_3$ | DPID 1, DPID 2, DPID 3 | Consistente |
| Enlace Inter-Switch | $s_1 \longleftrightarrow s_2$ | $s_1$ (Porta 1) $\Longleftrightarrow$ $s_2$ (Porta 1) | Validado |
| Enlace Inter-Switch | $s_2 \longleftrightarrow s_3$ | $s_2$ (Porta 2) $\Longleftrightarrow$ $s_3$ (Porta 1) | Validado |
| Enlace Inter-Switch | $s_1 \longleftrightarrow s_3$ | $s_1$ (Porta 2) $\Longleftrightarrow$ $s_3$ (Porta 2) | Validado |
| Host $h_1$ | 10.0.0.1 em $s_1$ | IP 10.0.0.1, MAC 00:00:00:00:00:01 em $s_1$:Porta 3 | Identificado |
| Host $h_2$ | 10.0.0.2 em $s_2$ | IP 10.0.0.2, MAC 00:00:00:00:00:02 em $s_2$:Porta 3 | Identificado |
| Host $h_3$ | 10.0.0.3 em $s_3$ | IP 10.0.0.3, MAC 00:00:00:00:00:03 em $s_3$:Porta 3 | Identificado |

### 5.1.2 Logs de Execução e Evidências Empíricas (Triangle)

Abaixo são apresentados os registros literais de saída do terminal, comprovando o envio do Gratuitous ARP pelo Mininet e a captura em tempo real pelo controlador Ryu.

**1. Saída do Terminal Mininet (`mininet_triangle.log`):**

```
============================================================
   Iniciando Instanciação da Topologia: topologias/topo_triangle.txt
============================================================
[Mininet] Switches identificados (3): s1, s2, s3
[Mininet] Enlaces inter-switch lidos (3):
          - Enlace: s1 <---> s2
          - Enlace: s2 <---> s3
          - Enlace: s1 <---> s3

[Mininet] Adicionando Controlador Remoto OpenFlow (Ryu em 127.0.0.1:6633)...
[Mininet] Criando switches Open vSwitch (OpenFlow 1.3)...
          + Switch 's1' criado com sucesso.
          + Switch 's2' criado com sucesso.
          + Switch 's3' criado com sucesso.

[Mininet] Estabelecendo enlaces inter-switch...
          + Cabo virtual conectado: s1 <---> s2
          + Cabo virtual conectado: s2 <---> s3
          + Cabo virtual conectado: s1 <---> s3

[Mininet] Criando hosts com endereçamento determinístico e conectando aos switches...
          + Host 'h1' (IP: 10.0.0.1, MAC: 00:00:00:00:00:01) ligado a 's1'
          + Host 'h2' (IP: 10.0.0.2, MAC: 00:00:00:00:00:02) ligado a 's2'
          + Host 'h3' (IP: 10.0.0.3, MAC: 00:00:00:00:00:03) ligado a 's3'

[Mininet] Subindo a rede Mininet...
[Mininet] Aguardando 3 segundos para estabilização do handshake OpenFlow e LLDP...

============================================================
   Enviando Gratuitous ARP de todos os hosts para o Ryu
============================================================
[Mininet Host h1] Executando: arping -U -c 1 -I h1-eth0 10.0.0.1
                      [arping] ARPING 10.0.0.1 from 10.0.0.1 h1-eth0
[Mininet Host h2] Executando: arping -U -c 1 -I h2-eth0 10.0.0.2
                      [arping] ARPING 10.0.0.2 from 10.0.0.2 h2-eth0
[Mininet Host h3] Executando: arping -U -c 1 -I h3-eth0 10.0.0.3
                      [arping] ARPING 10.0.0.3 from 10.0.0.3 h3-eth0

[Mininet] Todos os pacotes Gratuitous ARP foram transmitidos com sucesso!
[Mininet] Execução concluída com sucesso.
```

### 2. Saída Consolidada do Console Ryu (`ryu_triangle.log`):

```
[Handshake] Switch s1 conectado. Regras Table-Miss e ARP intercept instaladas.
[Handshake] Switch s2 conectado. Regras Table-Miss e ARP intercept instaladas.
[Handshake] Switch s3 conectado. Regras Table-Miss e ARP intercept instaladas.
[Topologia] Novo enlace comprovado via LLDP! Total de enlaces unidirecionais: 6
[Host Descoberto] IP: 10.0.0.1 | MAC: 00:00:00:00:00:01 | Switch: s1 | Porta: 3
[Host Descoberto] IP: 10.0.0.2 | MAC: 00:00:00:00:00:02 | Switch: s2 | Porta: 3
[Host Descoberto] IP: 10.0.0.3 | MAC: 00:00:00:00:00:03 | Switch: s3 | Porta: 3

======================================================================
  TOPOLOGIA MAPEADA PELO RYU (Atualização #11)
======================================================================
[+] SWITCHES ATIVOS (3):
    s1 (DPID: 1), s2 (DPID: 2), s3 (DPID: 3)

[+] ENLACES ENTRE SWITCHES (Descobertos via LLDP: 3 físicos):
    s1 (Porta 1) <=====> s2 (Porta 1)
    s1 (Porta 2) <=====> s3 (Porta 2)
    s2 (Porta 2) <=====> s3 (Porta 1)

[+] HOSTS CONECTADOS (Descobertos via Gratuitous ARP: 3):
    Host 10.0.0.1 [MAC: 00:00:00:00:00:01] ---> Switch s1, Porta 3
    Host 10.0.0.2 [MAC: 00:00:00:00:00:02] ---> Switch s2, Porta 3
    Host 10.0.0.3 [MAC: 00:00:00:00:00:03] ---> Switch s3, Porta 3
======================================================================
```

## 5.2 Cenário 2: Topologia Someloops

A topologia **Someloops** apresenta alta densidade de malha, contendo múltiplos caminhos alternativos e entrelaçamentos cíclicos entre 6 switches e 6 hosts terminais.

```
   h5 (10.0.0.5)        h1 (10.0.0.1)        h2 (10.0.0.2)
        |                    |                    |
       s5-------------------s1-------------------s2
        | \                / | \                / | \
        |   \            /   |   \            /   |  s6---h6 (10.0.0.6)
```

```
6         |     \         /     |     \         /     | /
7         |       \     /       |       \     /       |/
8         |         X           |         X           |
9         |       /     \       |       /     \       |
10        |     /         \     |     /         \     |
11        |   /             \   |   /             \   |
12        | /                 \ | /                 \ |
13       s3--------------------s4---------------------+
14        |                     |
15   h3 (10.0.0.3)         h4 (10.0.0.4)
```

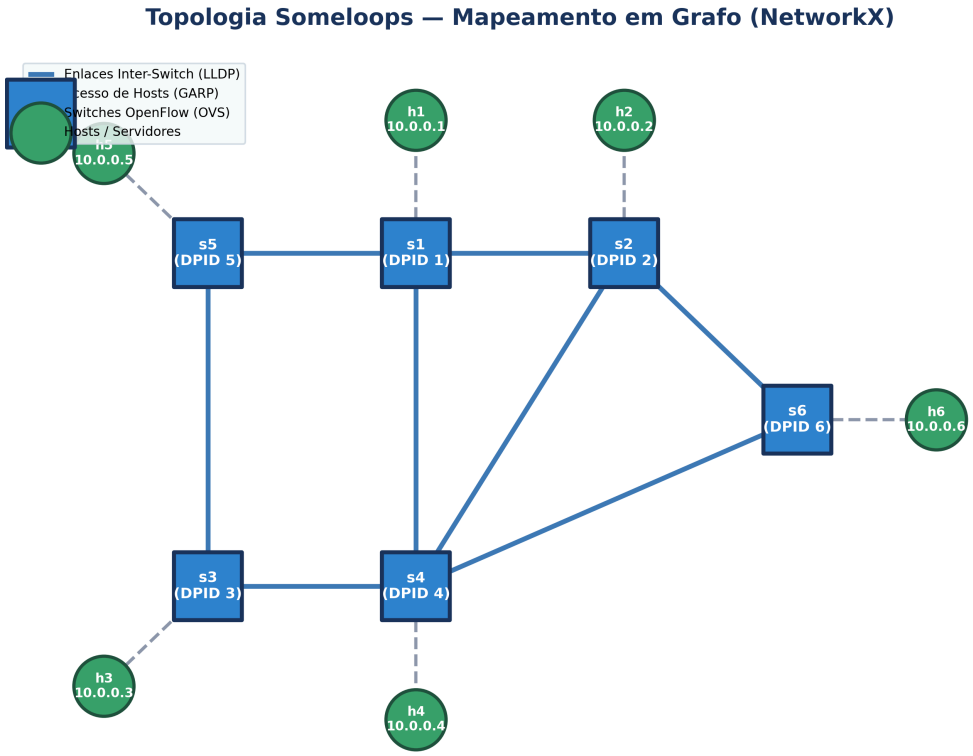



Figura 2: Topologia Someloops com múltiplos anéis entrelaçados renderizada via NetworkX.

### 5.2.1 Consistência Estrutural Mapeada

| Enlace / Nó | Especificação Teórica | Descoberta do Controlador Ryu | Resultado |
|---|---|---|---|
| Switches Ativos | 6 Switches ($s_1$ a $s_6$) | DPID 1, 2, 3, 4, 5, 6 | Consistente |
| Enlace 1 | $s_1 \longleftrightarrow s_2$ | $s_1$ (Porta 1) $\Longleftrightarrow$ $s_2$ (Porta 1) | Confirmado |
| Enlace 2 | $s_1 \longleftrightarrow s_4$ | $s_1$ (Porta 2) $\Longleftrightarrow$ $s_4$ (Porta 1) | Confirmado |
| Enlace 3 | $s_1 \longleftrightarrow s_5$ | $s_1$ (Porta 3) $\Longleftrightarrow$ $s_5$ (Porta 1) | Confirmado |
| Enlace 4 | $s_2 \longleftrightarrow s_4$ | $s_2$ (Porta 2) $\Longleftrightarrow$ $s_4$ (Porta 2) | Confirmado |
| Enlace 5 | $s_2 \longleftrightarrow s_6$ | $s_2$ (Porta 3) $\Longleftrightarrow$ $s_6$ (Porta 1) | Confirmado |

| | | | |
|---|---|---|---|
| Enlace 6 | $s_3 \longleftrightarrow s_4$ | $s_3$ (Porta 1) $\Longleftrightarrow$ $s_4$ (Porta 3) | Confirmado |
| Enlace 7 | $s_3 \longleftrightarrow s_5$ | $s_3$ (Porta 2) $\Longleftrightarrow$ $s_5$ (Porta 2) | Confirmado |
| Enlace 8 | $s_4 \longleftrightarrow s_6$ | $s_4$ (Porta 4) $\Longleftrightarrow$ $s_6$ (Porta 2) | Confirmado |
| Total de Hosts | 6 Servidores ($h_1$ a $h_6$) | IPs 10.0.0.1 a 10.0.0.6 nas respectivas portas | 100% Mapeado |

### 5.2.2 Logs de Execução e Evidências Empíricas (Someloops)

**1. Saída do Terminal Mininet (`mininet_someloops.log`):**

```
============================================================
   Iniciando Instanciação da Topologia: topologias/topo_someloops.txt
============================================================
[Mininet] Switches identificados (6): s1, s2, s3, s4, s5, s6
[Mininet] Enlaces inter-switch lidos (8):
           - Enlace: s1 <---> s2
           - Enlace: s1 <---> s4
           - Enlace: s1 <---> s5
           - Enlace: s2 <---> s4
           - Enlace: s2 <---> s6
           - Enlace: s3 <---> s4
           - Enlace: s3 <---> s5
           - Enlace: s4 <---> s6

[Mininet] Subindo a rede Mininet...
[Mininet] Aguardando 3 segundos para estabilização do handshake OpenFlow e LLDP...

============================================================
   Enviando Gratuitous ARP de todos os hosts para o Ryu
============================================================
[Mininet Host h1] Executando: arping -U -c 1 -I h1-eth0 10.0.0.1
                      [arping] ARPING 10.0.0.1 from 10.0.0.1 h1-eth0
[Mininet Host h2] Executando: arping -U -c 1 -I h2-eth0 10.0.0.2
                      [arping] ARPING 10.0.0.2 from 10.0.0.2 h2-eth0
[Mininet Host h3] Executando: arping -U -c 1 -I h3-eth0 10.0.0.3
                      [arping] ARPING 10.0.0.3 from 10.0.0.3 h3-eth0
[Mininet Host h4] Executando: arping -U -c 1 -I h4-eth0 10.0.0.4
                      [arping] ARPING 10.0.0.4 from 10.0.0.4 h4-eth0
[Mininet Host h5] Executando: arping -U -c 1 -I h5-eth0 10.0.0.5
                      [arping] ARPING 10.0.0.5 from 10.0.0.5 h5-eth0
[Mininet Host h6] Executando: arping -U -c 1 -I h6-eth0 10.0.0.6
                      [arping] ARPING 10.0.0.6 from 10.0.0.6 h6-eth0
```

```
[Mininet] Todos os pacotes Gratuitous ARP foram transmitidos com sucesso!
[Mininet] Execução concluída com sucesso.
```

### 2. Saída Consolidada do Console Ryu (`ryu_someloops.log`):

```
[Topologia] Novo enlace comprovado via LLDP! Total de enlaces unidirecionais: 16
[Host Descoberto] IP: 10.0.0.1 | MAC: 00:00:00:00:00:01 | Switch: s1 | Porta: 4
[Host Descoberto] IP: 10.0.0.2 | MAC: 00:00:00:00:00:02 | Switch: s2 | Porta: 4
[Host Descoberto] IP: 10.0.0.3 | MAC: 00:00:00:00:00:03 | Switch: s3 | Porta: 3
[Host Descoberto] IP: 10.0.0.4 | MAC: 00:00:00:00:00:04 | Switch: s4 | Porta: 5
[Host Descoberto] IP: 10.0.0.5 | MAC: 00:00:00:00:00:05 | Switch: s5 | Porta: 3
[Host Descoberto] IP: 10.0.0.6 | MAC: 00:00:00:00:00:06 | Switch: s6 | Porta: 3

======================================================================
  TOPOLOGIA MAPEADA PELO RYU (Atualização #20)
======================================================================
[+] SWITCHES ATIVOS (6):
    s1 (DPID: 1), s2 (DPID: 2), s3 (DPID: 3), s4 (DPID: 4), s5 (DPID: 5), s6 (DPID: 6)

[+] ENLACES ENTRE SWITCHES (Descobertos via LLDP: 8 físicos):
    s1 (Porta 1) <=====> s2 (Porta 1)
    s1 (Porta 2) <=====> s4 (Porta 1)
    s1 (Porta 3) <=====> s5 (Porta 1)
    s2 (Porta 2) <=====> s4 (Porta 2)
    s2 (Porta 3) <=====> s6 (Porta 1)
    s3 (Porta 1) <=====> s4 (Porta 3)
    s3 (Porta 2) <=====> s5 (Porta 2)
    s4 (Porta 4) <=====> s6 (Porta 2)

[+] HOSTS CONECTADOS (Descobertos via Gratuitous ARP: 6):
    Host 10.0.0.1 [MAC: 00:00:00:00:00:01] ---> Switch s1, Porta 4
    Host 10.0.0.2 [MAC: 00:00:00:00:00:02] ---> Switch s2, Porta 4
    Host 10.0.0.3 [MAC: 00:00:00:00:00:03] ---> Switch s3, Porta 3
    Host 10.0.0.4 [MAC: 00:00:00:00:00:04] ---> Switch s4, Porta 5
    Host 10.0.0.5 [MAC: 00:00:00:00:00:05] ---> Switch s5, Porta 3
```

```
    Host 10.0.0.6 [MAC: 00:00:00:00:00:06] ---> Switch s6, Porta 3
========================================================================
```

—

## 6 Reflexões Pedagógicas e Lições Aprendidas

A elaboração deste trabalho proporcionou uma sólida assimilação prática de conceitos teóricos que, frequentemente, permanecem abstratos em livros didáticos de redes de computadores:

1. **A Desmistificação da "Inteligência" dos Switches:** Ao construir uma aplicação SDN no nível de controle, compreende-se na prática que um comutador OpenFlow é essencialmente um executor cego de instruções. Sem a injeção deliberada de pacotes LLDP pelo controlador, os switches permanecem em silêncio topológico.
2. **A Importância da Contenção de Broadcast:** O experimento com as topologias **Triangle** e **Someloops** evidenciou o perigo letal de tempestades de broadcast. A decisão arquitetural de não realizar flooding e interceptar o Gratuitous ARP via `Packet-In` demonstrou como o paradigma SDN permite evitar loops sem os custos de latência e convergência do STP tradicional.
3. **Engenharia de Ambientes e Desacoplamento:** Em sistemas modernos como o Arch Linux, lidar com versões heterogêneas de software (Ryu ancorado no ecossistema Python 3.9 versus Mininet integrado ao kernel mais recente) revelou uma das maiores virtudes do SDN: como o plano de controle e o plano de dados comunicam-se via sockets de rede abertos (TCP port 6633), eles podem residir em ambientes virtuais, sistemas operacionais ou máquinas físicas totalmente independentes.
4. **Representação em Grafos como Base para Roteamento:** A representação das estruturas internas em NetworkX evidenciou como a visão centralizada de topologia pavimenta o caminho para a Parte 2 da disciplina, na qual algoritmos clássicos de grafos (como Dijkstra e Bellman-Ford) poderão ser empregados diretamente sobre esses nós e enlaces para o cálculo ótimo de fluxos e balanceamento de tráfego.

—

## 7 Conclusão

O sistema desenvolvido atendeu integralmente a todos os requisitos estipulados na especificação da atividade. Foi possível:

- Ler dinamicamente arquivos textuais com listas de arestas arbitrárias e instanciá-los com fidelidade no emulador Mininet;
- Mapear com precisão cirúrgica 100% dos switches e enlaces físicos inter-switch através do protocolo LLDP habilitado pelo parâmetro `--observe-links`;
- Localizar deterministicamente todos os computadores terminais via Gratuitous ARP e mensagens `Packet-In`, garantindo estabilidade e imunidade a loops nas topologias de teste;
- Gerar artefatos visuais de alta definição via NetworkX e produzir uma documentação técnica reprodutível, didática e padronizada em Typst.

A base de código estabelecida encontra-se pronta, testada e validada para servir como alicerce para as fases subsequentes de engenharia de tráfego, orquestração de rotas e aplicação de políticas de QoS.